

CS 305 Lab 1: Introduction to C, command line, and compiling

This lab introduces you to some of the basics of using the Unix/Linux commands and the vi text editor. You will also learn how to create, compile and run a C program using the command line.

For this lab, it is recommended that both partners log in, and that you each perform each task.

Objectives

Upon completion of the laboratory exercises, you will be able to do the following:

- Use Unix commands for directory navigation, creating directories, and opening files.
- Create, edit and save a text file using vi.
- Create, compile and run C programs using gcc.

Checkpoint 0 [30 points]: Hand in your prelab.

Part 1: Using commands

1. Log into the Engineering Linux VM. Create a Terminal by clicking on the 9-dots icon, , and then clicking on the Terminal icon. (You may need to scroll.)
2. You should see a screen with a cursor. Try typing `ls` (lower case L and lower case S) to list the contents of the current directory
3. **Getting help:** Let's see what the `head` command does. Type into the shell window:
`head --help`

This should show some documentation about the head command.

What does the head command do?

2. If you ever want additional terminal windows (which can be really handy), you can use the 9-dots icon and then right click on the Terminal and click on the New Window.

This will open another window where you can type commands, as you did in the main screen. In the new window, type:

```
less --help
```

3. Close the second xterm window and use the original one. Note that when you open a new terminal, the default directory is not necessarily your P-drive. You may need to navigate to the appropriate directory after launching a new shell window.

4. Before you used `ls` to see the content of the current directory. Now let's see what directory we are in:

```
pwd
```

It could say something like

```
/home/yourUserName
```

The slashes give you information about the path of the directory. The above would say that you are in the '*yourUserName*' folder, which resides in the home folder. From here, navigate to whichever folder you want to store your lab files. Use `cd` followed by the name of the folder to *change directory* into the given folder. For example:

```
cd P_Drive
```

Now list the contents:

```
ls
```

You should see files and folders that reside in this folder. Let's create a folder for this course:

```
mkdir CS305
```

Now, go into that directory:

```
cd CS305
```

Let's create a Lab1 directory:

```
mkdir Lab1
```

Go into that directory:

```
cd Lab1
```

If you need to move up one level of folders, you can use the command:

```
cd ..
```

Try that. Print working directory (`pwd`) to see that you are up one level. Now change back into the Lab1 directory (`cd Lab1`).

OK, you should now know how to make new folders and navigate to different folders in the shell. To see that you created new folders, you can open the P_Drive icon on the desktop, and then navigate to the CS305/Lab1 folder. This should be empty.

Part 2: Edit text with vi

1. The vi program is a standard text editor, which is what we will typically use to edit code. Once you become familiar with this tool, you can list it as a skill on your resume.

Vi commands (keyboard shortcuts for your reference)

Vi starts in the command mode. The positioning commands (e.g., cursor movement) operate only while vi is in command mode. You switch vi to input mode by entering any one of several vi input commands. Once in input mode, any character you type is taken to be text and is added to the file. You cannot execute any commands until you exit input mode. To exit input mode, press the escape (Esc) key. The following is a list of basic vi commands:

- Files.
 - vi filename open file in vi
 - :w save file
 - :wq save and exit vi

- :q! quit without saving changes
- Cursor movement. (Note: when using vi , you can also use the arrow-keys to move around.)
 - l move forward one character
 - h move back one character
 - ^ move to beginning of current line
 - \$ move to end of current line
 - j move down one line
 - k move up one line
 - :n go to line n
- Input commands (end with Esc)
 - a append after cursor
 - i insert before cursor
 - o open line below
 - O open line above
- Changes during insert mode
 - <ctrl>h move backward one character
 - <ctrl>w move backward one word
 - <ctrl>u move to the beginning of the file
- Cut/paste (input mode)
 - x delete a character
 - dd delete the current line
 - d delete to end of sentence (or line) from the cursor position
 - yy copy line of the current cursor position
 - p paste the deleted/copied character/line after the cursor position
- Window motions
 - <ctrl>d scroll down half a screen
 - <ctrl>u scroll up half a screen
 - /string search string forward
 - ?string search string backward
 - :set number show line number

2. Let's try vi now. In your `Lab1` folder in the shell, type:

```
vi test.txt &
```

- This should launch another window that looks like a text editor.
- If the window comes up split in two windows, you can cause the current pane to appear on the entire window with the C-x 1 command.

In the file, type:

```
This is a test file.
```

Then, save the file and quit vi. (see commands above for help).

4. Now, open the file in vi again. Make sure your previous contents were saved. (If not, go back and repeat the previous step.) Add your name to the top of the file, save it, and close it.

Part 3: Removing files & getting files for lab

1. If you ever want to remove a directory that is empty, you can use `rmdir` followed by the folder name.

BE VERY CAREFUL WHEN REMOVING FILES AND DIRECTORIES. There is no “undo” after a file or directory is deleted.

If you ever want to delete a file, you can use:

```
rm filename
```

If you need to remove all files within the current directory, you can use:

```
rm *
```

AGAIN, USING `rm` CAN BE DANGEROUS. For CS 305, you will likely not need to remove several files at once, so using `rm *` is not recommended.

2. If you ever want to re-issue a command, you can use the up-arrow to retrieve prior commands that you have typed. This will save you a bunch of typing if you use several of the same commands. If you went back too far, you can use the down arrow to check more recent commands.

- You can also modify the characters in these “recent commands” lines by typing characters to insert them, forward/backward arrow to move, backspace/delete to remove characters.

3. Open a web browser, for example, by opening the Firefox  icon. Go to learning.up.edu and choose the CS 305 course. Download `lab1.txt` and `sizes.c` from Moodle to your CS305/Lab1/ folder. Right-click and use “save link as” to save the files.

Open the file `lab1.txt` in `vi` or do a `less` on the file.

What is the special word of the day, located in the file? _____

Unix cheat sheet:

<code>cd xxx</code>	change directory to xxx
<code>cd</code>	change to home directory
<code>rmdir</code>	delete directory (must not have any other directories or files within it)
<code>rm</code>	delete file
<code>ls</code>	list contents in directory
<code>cp</code>	copy file/directory
<code>mv</code>	move file/directory
<code>mkdir</code>	make a new directory
<code>pwd</code>	print working directory

Part 4: Compiling and Running C Programs

We will use the `gcc` command to compile a C program. For example, to compile `my_prog.c`, into an executable called `a.exe` we write:

```
gcc -g my_prog.c
```

The `-g` indicates that debugging information should be produced. The debugger information will be useful when we get the point of using the debugger in the course. This creates an executable file with the name `a.out`. This is the default executable file name for a C program in Linux.

To run the program, you simply type at the command prompt: dot, forward slash, and the name of the executable file like so: `./a.out`, followed by the return (or enter) key. When you run the program, it will begin executing the main function which should be declared as:

```
int main(int argc, char* argv[]) { ... }  
(or)  
int main(int argc, char** argv) { ... }  
(or)  
int main(void) { ... }
```

1. Type the command to compile `my_prog.c`. Note, it is expected that you don't have a file called `my_prog.c`:

```
gcc -g my_prog.c
```

What message did you get? _____

Why did you get that message? _____

2. In your `Lab1` folder, use `vi` to create a new C program for printing "Hello, World!". The program should be named `hello.c`.

```
vi hello.c
```

Type the following code into the file:

```
#include <stdio.h>  
  
/* main program prints Hello, world */  
int main(int argc, char* argv[]) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

3. Compile and run the program in the command-line window. (Make sure you save the file before you compile.)

```
gcc -g hello.c  
./a.out
```

4. Modify the program so that it has your name in comment at the top of the file. Modify the program so that it prints "I love C" 20 times (each on a different line) after printing "Hello, world!" once. A for-loop in C is just like a for-loop in Java:

```

    for(int i = 0; i < 10; i++) {
        /* body */
    }

```

5. Save the file, compile it, and run your modified program. Note: Each time you make a change to the program, you must re-compile it for the changes to take effect.

Checkpoint 1 [25 points]: Let your lab instructor/assistant see the modified "Hello world" running and your answers to the questions up to this point in the lab.

Part 5: Command line arguments

In C programs, the main function has two parameters: `argc` and `argv`. The first, `argc`, is the number of command line arguments, including the program executable name. The second, `argv`, is the array containing those command-line arguments as strings. In `argv`, `argv[0]` is the name of the program, `argv[1]` would be the next item, and so forth. For example, if there is a program called `my_prog`, then the execution of:

```
my_prog tammy sally fred
```

would mean that `argv[0]` = "my_prog", `argv[1]` = "tammy", `argv[2]` = "sally" and `argv[3]`="fred" and `argc` = 4.

1. Copy `hello.c` to `command_line.c`:

```
cp hello.c command_line.c
```

Open `command_line.c` in vi:

```
vi command_line.c &
```

Remove the printing of "hello". Instead of always printing "I love C" 20 times, update the code to do the following:

- If `argc` is not equal to 2:
 - Print "I love C!" followed by a newline. (print just once)
- Else:
 - Print the argument after the name of the program given on the command line followed by a newline (print just once)

```

if(argc != 2) {
    printf("I love C!\n");
} else {
    printf("%s\n", argv[1]);
}

```

Note that the syntax for conditions (if and if/else) in C is the same as Java. Now that we have more than one program in the directory, it makes more sense to name the programs when we compile them instead of using the default `a.out`. When you compile `command_line.c`, now use the form with the '-o' switch:

```
gcc -g command_line.c -o command_line
```

2. Run the code with the following calls:

```
./command_line  
./command_line apple  
./command_line apple banana
```

What does each of these print?

Checkpoint 2 [25 points]: Let your lab instructor/assistant see the output of the program executions and your answers above.

Part 6: Variable types and sizes

Let's experiment with variables, their types, and the amount of space they take in memory.

1. Go to your `Lab1` folder if you are not already there. If you have not already done so, download `sizes.c` from Moodle by right-clicking and choosing Save As. Open `sizes.c` in `vi` and read through the code. This program assigns different types of variables, pointers to variables, prints the sizes, and prints the values.

2. Compile and run the code. Name the executable `sizes`.

How many bytes are used to store ints? _____

How many bytes are used to store chars? _____

How many bytes are used to store doubles? _____

How many bytes are used to store pointers? _____

3. Re-compile with optimizations by running (it's a capital letter O below, not a zero):

```
gcc -g -O -o sizes sizes.c
```

Re-run the code a second time:

```
./sizes
```

Did the values for the int, char, and double variables change? Yes No

Did the values for the pointers change? Yes No

Why would the values for the pointers change? _____

4. Modify the code to add one new type of variable throughout the program. C supports several types, such as short, long, unsigned int.

What is the new type you added? _____

How many bytes are used to store this type? _____

NOTE THAT NOT ALL C PROGRAMS HAVE THE SAME SIZES FOR TYPES. In Java, across all platforms, the size of an integer is always the same (4 bytes). In C, it is likely 4 bytes, but it could be defined for a certain architecture to be as low as 2 bytes—or it could be something like 8. The spec states that chars are at least 1 byte, ints at least 2 bytes. Also, the `sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`.

C also has types that include signed and unsigned. If the type is `int` (without `unsigned` in front), it is defaulted to be a signed `int`. An unsigned `int` only stores non-negative values. Feel free to experiment with creating signed, unsigned, and regular types to see what data they can store.

Checkpoint 3 [20 points]: Let your lab instructor/assistant see the output of the modified program and your answers above.

Part 7: Note on Printing Code

1. If you need to print your code, you may want to open move it to another computer in order to print it.

2. Log out from the Engineering Linux machine, which can be done by pressing the  icon near the top-right; then click your user name; then “Log Out”. You are finished.

FYI: Unix commands on Mac or PC

The Unix commands you have learned in this lab can be applied directly in xterm windows on a Mac or Linux computer.